SimpleChef: Progress Report
Human–Computer Interaction (CS4063)

[Group Name]
[Institution / Course Section]
Email: [contact email]

***Abstract*—This progress report summarizes development of SimpleChef, a mobile-first cooking assistant built with React Native (Expo), FastAPI, and PostgreSQL. We describe objectives aligned with our project proposal, implemented features as of March 28, 2026, including a controller-based frontend architecture that separates application logic from presentation for alignment with Figma-first UI work. We reference consolidated repository documentation (requirements traceability, API contract, Figma UI system requirements, and backend refinement notes) rather than duplicating those specifications here. Implementation evidence shows high-fidelity phone-aspect UI designs (Section V); these screens are not yet wired to the live API in the design tool, but the backend endpoints exist for each flow, and connection is planned through the existing controller hooks. We summarize ongoing gaps, remaining work, workflow and architecture figures, and an updated milestone timeline toward final delivery.**

I. Project Overview

SimpleChef is a mobile-first cooking assistant designed to reduce friction across the entire cooking process—from discovering a recipe to meal planning and grocery shopping. The tagline *Your calm kitchen companion* reflects the goal of a capable, calm tool that streamlines everyday cooking tasks.

As stated in our project proposal, many cooking apps fragment the experience: recipes in one place, calories in another, grocery lists on paper or in notes. Instructions often appear as long blocks of text, and timers frequently push users out of the recipe context. SimpleChef targets a single cohesive flow: recipe discovery, step-by-step cooking with integrated timers, meal planning with calorie awareness, and grocery support. The design applies HCI ideas including progressive disclosure, clarity over clutter, and smart defaults with full user control [1], [2].

A. Objectives (aligned with the proposal)

- **Unify the workflow** across browsing, cooking mode, planning, and groceries with consistent navigation and data flow.
- **Optimize step-by-step cooking** so each step is scannable during active cooking, with integrated timers and mise en place tied to steps where modeled.
- **Timer management** via a timer dock that surfaces active timers without leaving the cooking context.
- **Grocery automation** from planned meals (deduplication, categories, merge from plan, export).
- **Lower recipe input friction** via manual entry plus AI-assisted text parsing with verification before save; image and video import remain planned.
- **Meal planning and calorie goals** by assigning meals to days, day-level calorie summaries vs. profile goals, and recipe filters (API); richer statistics are extensions.
- **Iterate with users** through feedback sessions and refinement of flows and visuals [3].

B. Tools and technology

- **Frontend:** React Native (Expo). Routes under `frontend/app/`; **application logic** is centralized in `frontend/controllers/` (hooks such as `useRecipeLibraryController`, `usePlannerController`, `useGroceryController`) so screens stay presentation-focused and can be mirrored in Figma without duplicating API rules. Shared DTOs live in `frontend/types/`; HTTP and auth live in `frontend/services/api.ts`.
- **Backend:** FastAPI (Python) with SQLAlchemy models and Alembic migrations; JWT auth for protected routes (`backend/app/api/`).
- **Data store:** PostgreSQL (via Docker Compose for local development).
- **Design:** Figma for wireframes and high-fidelity exploration [4]; repository documentation (Section II) lists every named UI/API requirement for annotation without duplicating them in this PDF.
- **AI/ML:** Text parsing uses a structured placeholder/demo path (`backend/app/services/ai_service.py`) with validation (`recipe_parse_validation`); production LLM/vision is not required for the current milestone.

II. Repository documentation (single source of truth)

Detailed specifications are maintained under `docs/` and `frontend/` in the repository. **The PDF does not duplicate them**; cite or link these paths in appendices and team notes.

III. Project Progress Overview

As of March 28, 2026, the team has working vertical slices: an Expo client, FastAPI API, and PostgreSQL database support authenticated use of the main tabs and core flows described in the proposal. The frontend follows a **controller layer**: route files bind UI to hooks that encapsulate loading state, validation, and service calls.

TABLE I
KEY DOCUMENTATION FILES (RELATIVE TO REPOSITORY ROOT).

Artifact	Purpose
Figma UI requirements	Global UX, screen tables, consolidated checklist (proposal + Prompt + SRS + checklist), data fields, states, deferrals. Path: `docs/FIGMA_UI_...`.
Traceability matrix	SRS/proposal → endpoints and status. `docs/REQ_...`.
API contract	Query params, filters, planner day summary. `docs/API.md`.
Architecture	Auth/ownership, **frontend layering**. `docs/ARCHITECTURE.md`.
Backend refinement	Design debt and suggested refactors. `docs/BACKEND_...`.
SRS & backlog	`docs/REQUIREMENTS.md`, `CONTINUATION_CHECKLIST.md`.
Frontend guides	Controller pattern, design tokens. `frontend/README.md`, `frontend/DESIGN_SYSTEM.md`.

A. Completed milestones and components

- **Proposal and design specification:** Scope, problem statement, five-tab UI (Home, Calendar, Add, Grocery, Profile), wireframe, and timeline documented.
- **Backend foundation:** Application entry (`main.py`, `run.py`), models for users, recipes (ingredients, steps), meal plans, grocery lists/items, and versioned migrations.
- **Authentication:** Signup and login; JWT via Axios interceptor (`frontend/services/api.ts`); recipe visibility and ownership enforced server-side (see `docs/ARCHITECTURE.md`).
- **Recipe library (Home):** Debounced search, difficulty chips, `RecipeList` fed by `useRecipeLibraryController`; API supports additional filters (`max_total_minutes`, `tags_all`) for future UI.
- **Recipe detail:** Hero image or placeholder, summary stats, expandable ingredients, steps; **Begin Cooking**; add to calendar modal; owner edit/delete.
- **Cooking mode:** Step navigation, progress, mise en place per step (ingredient–step linking), timer dock (`TimerDock`, `useTimerStore`), optional per-step timers, "all ingredients" modal, keep-awake when profile requests it.
- **Add recipe:** Manual form with dynamic rows; paste flow calls `POST /recipes/parse` and opens verification on the manual screen. Image scan remains disabled (*Coming Soon*).
- **Meal planner:** Month grid, selected day, meals list, quick add, add from recipe library with meal type, delete; `GET /planner/day-summary` supports consumed calories vs. profile goal on the calendar screen.
- **Grocery list:** Sectioned list, checkbox sync, manual add, **merge from meal plan** (date range), share/export text.
- **Profile:** Edit name, bio, calorie goal, dietary restrictions (comma-separated), keep-screen-on toggle; save via `PATCH /users/me`; friends/social explicitly out of scope for this build.

B. Gaps relative to the proposed UI and SRS

- **Home:** Filter drawer (cook time, profile-default dietary tags) not fully exposed in UI though API supports several query parameters.
- **Cooking mode:** Prompt/proposal describe replacing the bottom tab bar with cooking-specific chrome; current app keeps stack navigation inside the tab shell—documented as a delta in `docs/FIGMA_UI_SYSTEM_REQUIREMENTS.md`.
- **Calendar:** Month indicators (dots/images per day), color coding by meal type, and statistics charts are not implemented.
- **Grocery:** Full inline editor and drag-to-reorder categories are not implemented; merge and check-off work.
- **AI parsing:** Demo/placeholder parser, not a production LLM or vision pipeline.
- **Friends / social:** Out of scope per backlog; proposal text remains for future product direction.

IV. ONGOING CHALLENGES AND DESIGN DIFFICULTIES

A. Closing the loop among meal plans, recipes, and groceries

The proposal promises a grocery list grounded in planned meals. A `POST /grocery/from-plan` merge path aggregates ingredients from planned recipes in a date range; users can still edit items manually. Further polish: servings scaling rules, richer unit normalization, and optional source-recipe tags in the list UI.

B. Search, filter, and dietary personalization

Server-side filters exist; the Home UI should expose time caps and tag filters aligned with profile dietary strings to limit choice overload on Home [2].

C. Mise en place and step granularity

Ingredients can link to steps; the cooking screen shows global prep on step one and step-linked items thereafter. Parsing quality and editor UX for linking remain improvement areas.

D. Trustworthy AI parsing

A mock parser can mislead participants about accuracy. A real text parser behind the same verify-and-edit screen, with image/video clearly labeled beta, remains a priority [3].

E. Calendar information density

Month grid exists; richer at-a-glance indicators and proposal-style bottom sheet are partial. Usability tests should compare taps-to-complete for "add from library."

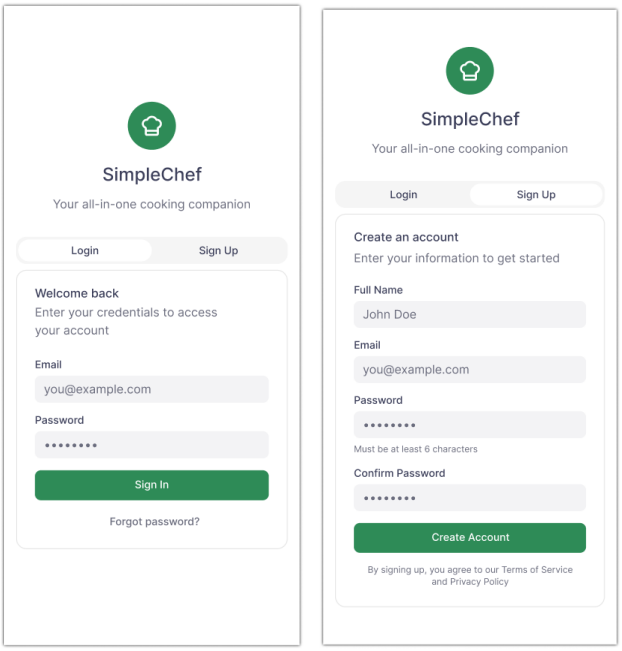
F. Engineering and maintainability

Cross-cutting concerns (authorization, validation boundaries, service vs. router responsibilities) are recorded in `docs/BACKEND_REFINEMENT_NOTES.md` for future refactors without blocking feature delivery.

V. Implementation Evidence

The following figures are **high-fidelity UI designs** (phone aspect ratio) captured for this report. They are organized by feature area (e.g. profile, login, home, etc.). These designs are not yet connected to the running backend in the main repo as there is still some refinement to be done, but the backend is set up with specific exposed APIs so it is very easy to connect and modify the frontend. **For every screen shown below, a corresponding backend already exists** in the repository (e.g., auth, recipes, planner, etc.) which is all documented in our shared repository. Because of this no new architectural pattern is required, all we have to so is write each view to the appropriate controller and DTO types, which is much more simple than having to rewrite large connections.

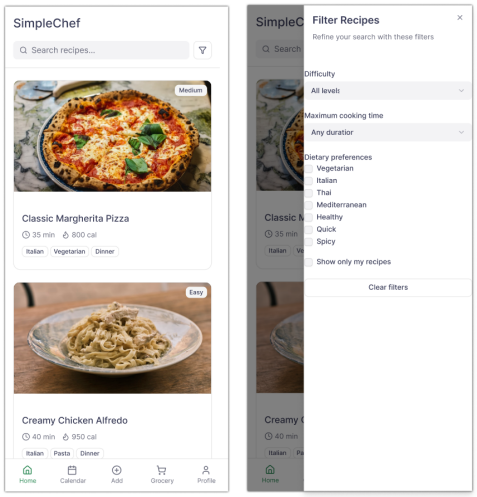
Authentication (sign-in and registration) - This page has two main sections, one for returning users and the other for new users. Users will remain logged in when the app is closed, but when they are using a new device they will have to log in still, hence why a login screen is present. For new users there is a page to create an account, which only requires the simple info typical of a login screen. In the future, we may add a next step to this new user account creation that will consist of a walkthrough of the app's functions and a prompt to enter other account details (e.g. goals, dietary restrictions, etc.), making this lead into onboarding rather than just throwing new users into the app.



Login Create account

Fig. 1. Authentication: email/password login and account creation.

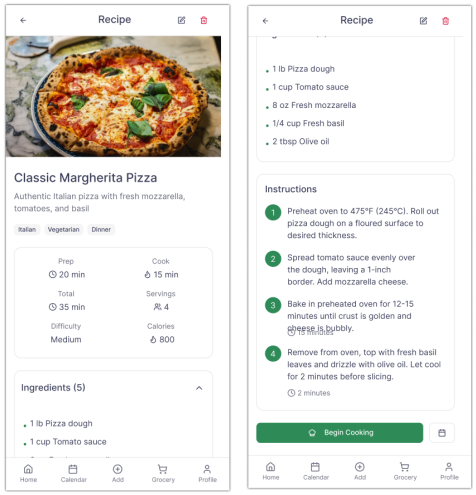
Home (recipe library) - This page is the home screen which consists of a list of recipes viewable by scrolling, this list can be filtered by a variety of tags which can be applied to recipes for easier searching. These tags are a stand in for now, but in later implementations they will likely be based on tags already one existing food so there is no redundancy.



Home (library) Search and filter chips

Fig. 2. Home screen: List of recipes and filter menu.

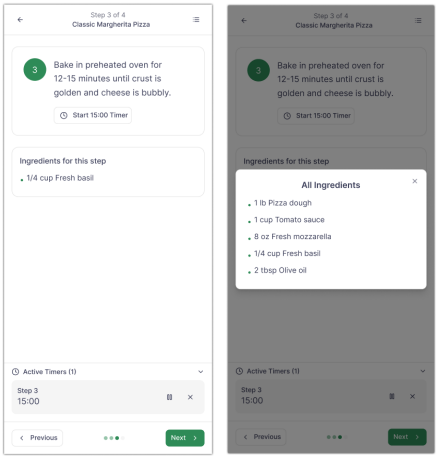
Recipe detail - The following is what a user sees when a recipe is clicked, it contains a short overview of the recipe along with everything the user would need before starting the recipe. The user can also edit and remove this recipe from this submenu with the icons in the top right, and go back to the home screen with the arrow in the top left. The edit menu is the same as the recipe creation menu which will be seen shortly.



Recipe detail (top) Recipe detail (bottom)

Fig. 3. Recipe detail (scroll): summary, ingredients, and actions. Controller: `useRecipeDetailController`. API: `GET /recipes/{id}`.

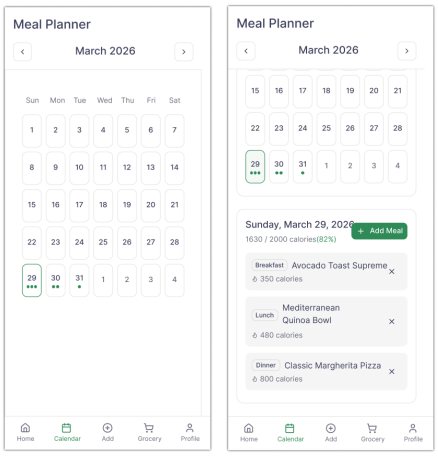
Help / instructions - The following is still very much a work in progress, but it is the guided recipe screen a user will see when a recipe is started. There are buttons at the bottom to see progress and go forward or back, along with an indicator of progress. The timer button is visible in the instructions and when started appears at the bottom, which can be used to pause or cancel the timer. The button in the top right can be selected to see all the ingredients for the recipe. This is one of many iterations of these menus, and there are likely going to be a few more until we land on a final product.



Instructions (full screen) Instructions (modal)

Fig. 4. Onboarding or help for library search and filters (draft labels). Not tied to a single API route; supports discoverability on Home.

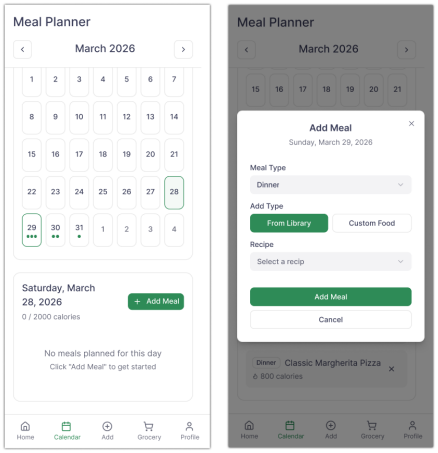
Calender - This page contains the calendar for the month, and is the main way to plan recipes/meals for the user. Days can be selected by clicking on them, and the current day is selected by default. When meals are added to a day they are shown via dots on the selected day, and when the day is selected the meals and calories are shown.



Month grid Day selected

Fig. 5. Calendar: month overview and highlighted date.

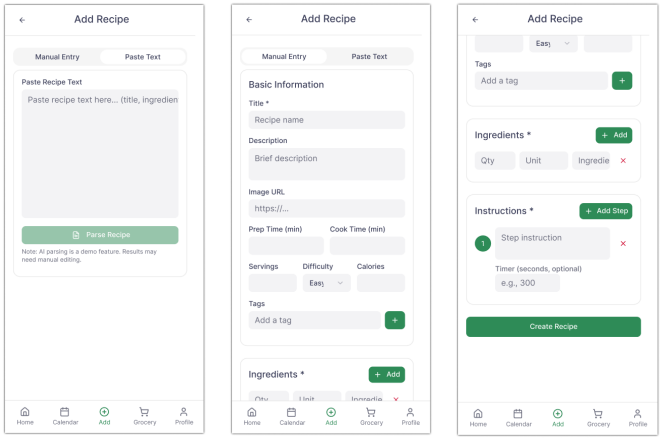
Empty day/Adding meal - The first image is what appears when a day with nothign planned is slected, prompting the user to select a meal. The second image is the modal that appears when the add meal button is selected providing the options to select a recipe from the user library, or to input a custom food (e.g. a snack, or a purchased meal).



Empty day Add meal (planner modal)

Fig. 6. Planner: empty day and add-meal dialog.

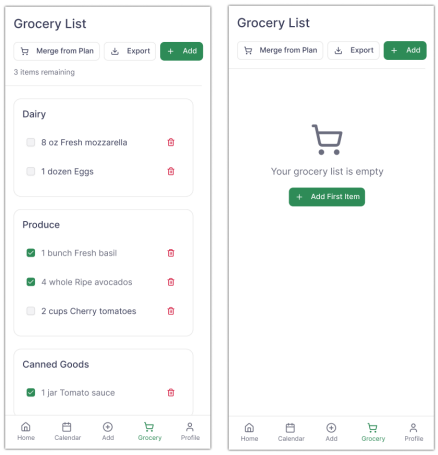
Add recipe - This is where a user will input a recipe, right now there are only two options, but hopefully in the future we can add more options to input recipes. The main and only functional method right now is manually entering the recipe which is done with the latter 2 images. The first image is the soon to be implemented ai parsing of a recipe which will (hopefully) use AI/ML to parse inputed text into a recipe that can then be edited and saved.



Add recipe (paste / AI) Manual entry (bottom) Manual entry (top)

Fig. 7. Add recipe: import entry point and manual create/edit form. Controllers: `useAddRecipeController`, `useManualRecipeController`. API: `POST /recipes/parse`, `POST /recipes/`, `PUT /recipes/{id}`.

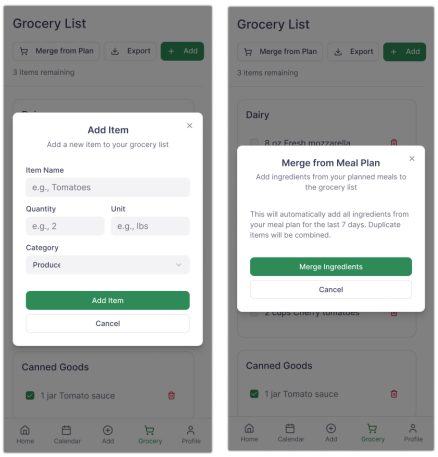
Grocery list—main views. Sectioned list with items and checkboxes versus an empty-state prompt before items exist.



Grocery list Empty grocery

Fig. 8. Grocery list: populated categories and empty state. Controller: `useGroceryController`. API: `GET /grocery/`.

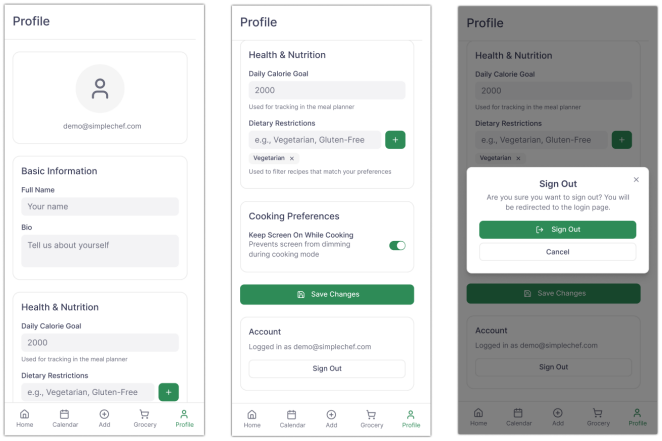
Grocery—modals. Manual line-item entry and merging ingredient demand from the meal plan into the list for a date range.



Add grocery item Merge from plan

Fig. 9. Grocery: add-item sheet and merge-from-plan confirmation. API: `POST /grocery/items`, `POST /grocery/from-plan`.

Profile and sign-out. Editable goals and preferences (scrolled), and a confirmation when ending the session.



Profile (top) Profile (bottom) Sign out confirmation

Fig. 10. Profile: account fields, goals, and dietary options; sign-out dialog. Controller: `useProfileController`. API: `GET/PATCH /users/me`.

Optional: include a repository URL or short demo video link in the final PDF footer or abstract footnote.

VI. Future Work and To-Do

Remaining work is grouped by horizon. Table II lists near-term tasks; Table III covers mid-term polish; Table IV lists course deliverables.

TABLE II
Near-term tasks (late March–April).

Task	Notes
Expose Home time/tag filters in UI	API ready; align with profile dietary default
Month calendar indicators + statistics	Dots/trends per proposal; optional charts.
Replace mock AI parser	Keep verify-and-edit UX; improve extractio
Figma high-fi + navigation delta	Cooking chrome vs. tabs; see Figma require

TABLE III
Mid-term tasks (proposal weeks 5–7 and polish).

Task	Notes
Image / video import	OCR or model pipeline; loading and failure
Grocery inline edit + reorder	Persisted ordering if required by product.
Optional statistics	Trends vs. goals using meal logs.
Friends / sharing	If in scope: APIs and UI.

TABLE IV
Course deliverables tied to evaluation and documentation.

Task	Notes
Usability testing	Task scripts: browse, cook with timers, plan
Heuristic and accessibility pass	Contrast, touch targets, legibility while cook
Final demo and report	Live demo, design rationale, user analysis p

VII. Project Plan and Concept

Figure 11 summarizes primary user journeys in the current build; dashed arrows indicate partial or planned flows. Figure 12 shows the high-level system context.

VIII. Project Progress Timeline

Table V summarizes major phases and status. Dates are illustrative for Spring 2026; adjust to your group's actual calendar.

a) Risks: Parser quality and advanced planner/grocery UX depend on continued parallel API work with Figma prototypes; documentation in `docs/` reduces rework between design and implementation.

IX. Conclusion

SimpleChef has moved from design specification to a **functional prototype** with a documented separation between **presentation** (Expo routes and components) and **application logic** (controller hooks), backed by a traceable API and SRS mapping. The **figures in Section V** document the intended high-fidelity UI; where those designs are not yet connected to the API, **backend support and controller wiring** provide a direct path to integration. Authenticated users can browse and filter recipes (with further UI exposure planned), inspect detail, run step-by-step cooking with a timer dock and mise en place, add recipes manually or via parse-then-edit, plan meals on a monthly calendar with day-level calorie summaries, merge groceries from plans, and edit profile settings. Remaining gaps versus the full proposal include richer Home filters in the UI, calendar visual density and statistics, production-grade parsing, grocery inline editing and reorder, optional social features, and evaluation activities (usability testing, heuristic review, accessibility) ahead of final delivery. All named requirements for Figma and implementation are consolidated in `docs/FIGMA_UI_SYSTEM_REQUIREMENTS.md` and `docs/REQUIREMENTS_TRACEABILITY.md`.

References

[1] J. Preece, Y. Rogers, and H. Sharp, *Interaction Design: Beyond Human-Computer Interaction*, 4th ed. Wiley, 2015.

[2] R. Hartson and P. S. Pyla, *The UX Book: Process and Guidelines for Ensuring a Quality User Experience*. Morgan Kaufmann, 2012.

[3] J. Nielsen, *Heuristic Evaluation*. John Wiley and Sons, 1994.

[4] "Figma help: Prototyping and design," https://help.figma.com/hc/en-us/categories/360002051614-Design, accessed March 2026.

[5] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," https://www.w3.org/TR/WCAG21/, accessed March 2026.

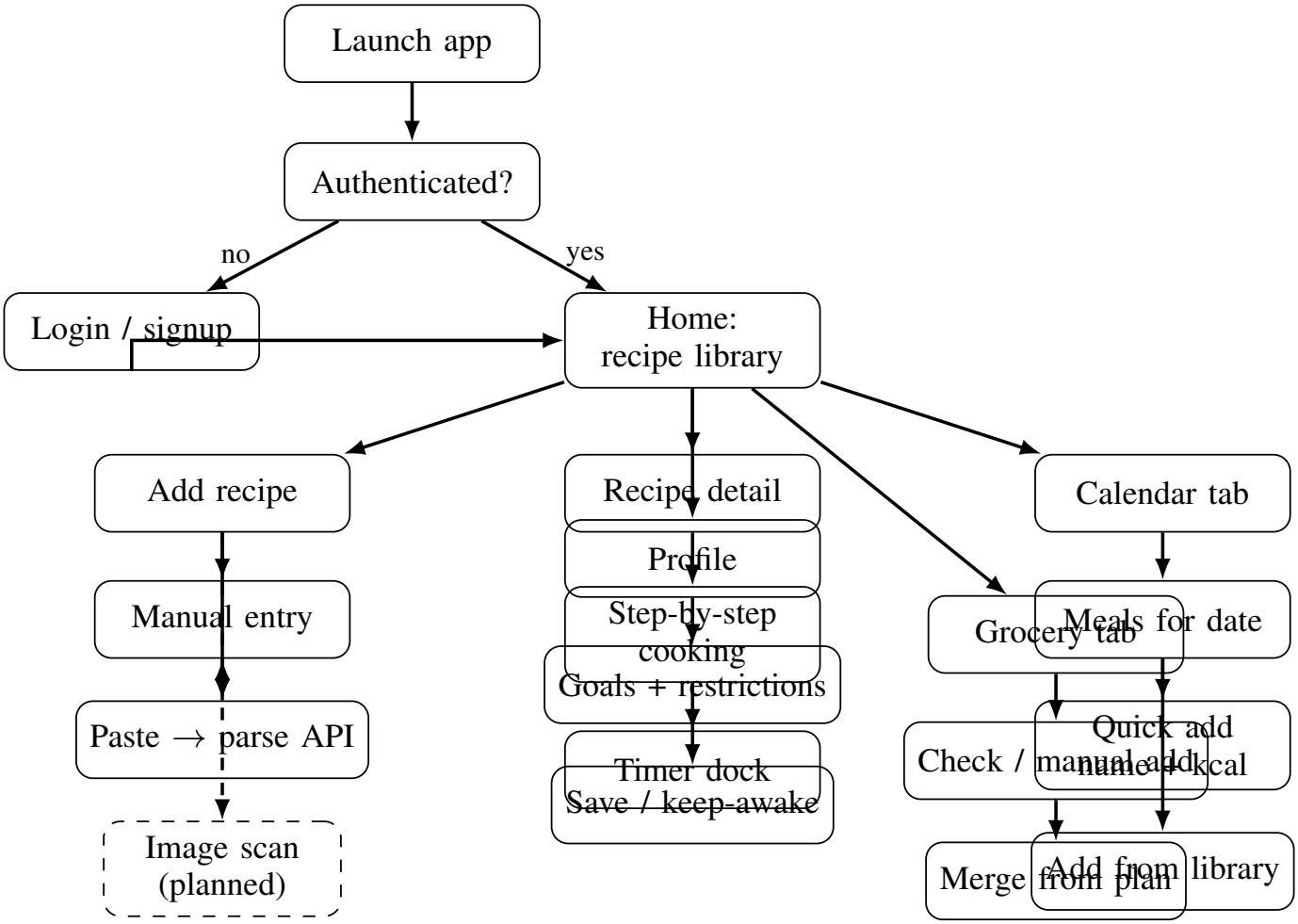


Fig. 11. User flow emphasizing implemented paths; dashed nodes are planned or partial.

TABLE V
MILESTONE SUMMARY (SPRING 2026, INDICATIVE DATES).

Phase / task	**Status**	**Span (approx.)**	**Notes**
Proposal and wireframe	Done	Feb 1–14	Scope and specs
Repository docs (Figma req., traceability, arch.)	Done	Mar 20–28	Single sources under `docs/`
Frontend controller layer	Done	Mar 20–28	`frontend/controllers/`
Backend API and database	Done	Feb 15–Mar 7	Models, migrations, routers
Auth and profile	Done	Feb 20–Mar 28	JWT, `/users/me`, PATCH profile
Recipe library and detail	Done	Feb 25–Mar 10	List, detail, filters (API)
Cooking mode and timer dock	Done	Mar 1–Mar 14	Steps, mise, timers
Add recipe (manual + parse)	Done	Mar 5–Mar 15	Verify-and-edit path
Meal plan API and calendar UI	Done	Mar 8–Mar 28	Grid, library add, day summary
Grocery list + merge from plan	Done	Mar 10–Mar 28	Sections, merge, share
Figma high-fi + filter UI parity	Active	Mar 22–Apr 10	Match `FIGMA_UI_...`
Real AI text parsing	Planned	Mar 25–Apr 8	Replace mock service
Month indicators + statistics UI	Planned	Apr 1–Apr 20	Proposal polish
Usability tests on build	Planned	Apr 5–Apr 15	Scripted tasks
Heuristics and accessibility	Planned	Apr 12–Apr 20	WCAG-oriented fixes [5]
Final demo preparation	Planned	Apr 18–Apr 28	Presentation and polish
Final documentation deadline	Milestone	May 3	Final report (syllabus)

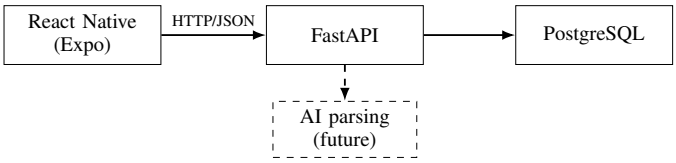


Fig. 12. System context: mobile client, API layer, database; optional AI service for parsing.